DATA SCIENCE PROJECT WHITEPAPER

Automated web extraction, unsupervised customer segmentation, and time-series forecasting with a model interpretability layer

Scope: Tasks 2-4 from the provided Data Science Tasks & Instructions. Task 1 is intentionally excluded in line with the earlier request.

Executive summary

This package demonstrates a reproducible data science workflow. Task 2 supplies a polite BeautifulSoup scraper and a parser test fixture. Task 3 uses standardized behavioral metrics, K-Means diagnostics, and PCA to create four illustrative customer segments. Task 4 evaluates stationarity, fits a seasonal SARIMAX model, forecasts twelve months with 95% intervals, and uses SHAP to interpret an auxiliary one-step Random Forest model.

1,200 synthetic customer rows 4 clusters	**Oct 2026 - Sep 2027** 12 monthly periods 95% confidence intervals	**3.63% MAPE** 12-month holdout SARIMAX demonstration

DATA INTEGRITY: no real customer or historical business data was supplied. Tasks 3-4 use synthetic demonstration records. The scraper code is complete, but live web retrieval could not be performed in this build environment; bundled parser rows are explicitly marked as offline test fixtures, not real scraped data.

Deliverable files

Python scraper and analysis pipeline, an interactive Streamlit dashboard, Jupyter notebook, generated CSV outputs, diagnostics, chart images, and this six-page whitepaper are included in the project archive.

Prepared: October 2026 | Reproducible demo build

2 | Task 2 - Automated web scraping and extraction

Objective: collect public text listings from an unstructured HTML source, extract core fields with text-selection logic, clean whitespace, and save the result to CSV and a database.

Implementation

Source target: Quotes to Scrape (https://quotes.toscrape.com/), a public practice site intended for scraping exercises. The script uses Requests with retries and a descriptive user-agent. It follows pagination links, applies a configurable delay, and stops at a configurable maximum number of pages.

Parsing and cleaning: BeautifulSoup selects each `div.quote`, reads the quote text and author, extracts all tags, replaces non-breaking and zero-width spaces, collapses repeated whitespace, rejects blank/short/unusable text, computes character and word counts, then removes duplicate quote-author pairs.

Output schema

Field	Definition	Treatment
quote	Main text listing	Whitespace normalized; empty text excluded
author	Attributed author name	Whitespace normalized; required for record
tags	Page tags joined by |	Blank tags omitted
source_url	URL of page captured	Retained for lineage
scraped_at_utc	Capture timestamp	UTC ISO timestamp
quote_char_count / quote_word_count	Derived text metrics	Calculated after cleanup

Storage and reproducibility

A successful live run writes `quotes_scraped.csv` and `quotes_scraped.sqlite`, with records stored in the `quote_records` table. A local HTML fixture and parser test verify extraction offline. The included fixture export contains only made-up demo rows with source_url set to example.invalid; it must not be cited as live-collected data.

Run from the project folder: python task2_web_scraper.py --pages 5 --output-dir outputs/task2_live_scrape --delay 1. Internet connectivity is required. Check terms of service and robots policies when changing the target; do not bypass access controls.

3 | Task 3 - Unsupervised customer segmentation

The demonstration uses 1,200 synthetic customer records with demographic, purchase-frequency, order-value, channel and return-rate metrics. A small number of missing online-ratio values are intentionally present in the raw file to demonstrate data preparation.

Pipeline

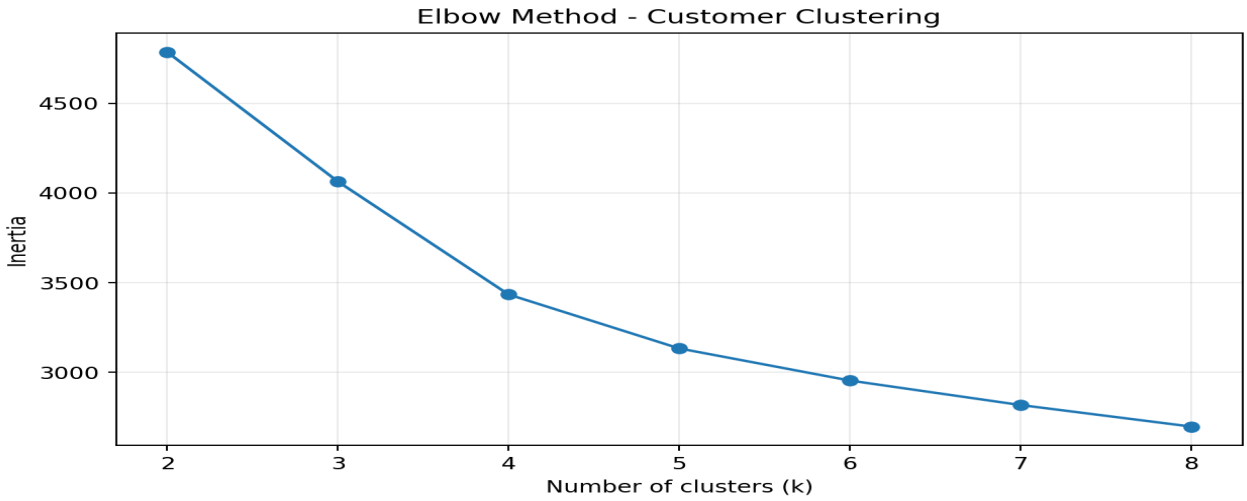
1. Data quality: remove duplicate customer IDs; coerce feature columns to numeric; median-impute missing numeric values; clip numeric values to the 1st and 99th percentiles to reduce the influence of extreme values. In real data, compare such treatments with domain limits and robust methods before clipping.

2. Standardization: apply StandardScaler to age, annual income, spending score, monthly purchase frequency, average order value, online purchase ratio and return rate. Standardization is important because the raw metrics have different units and ranges.

3. Cluster diagnostics: fit K-Means for k=2 through k=8, recording inertia for the elbow method and silhouette score for cohesion/separation. The demo selects k=4 because the synthetic data was intentionally generated from four latent profiles; this is not evidence that a real customer dataset should use four clusters.

4. Projection: PCA maps the standardized feature matrix into two principal components for visual inspection. PCA is a visual reduction, not a replacement for cluster validation in the full feature space.

Elbow diagnostic

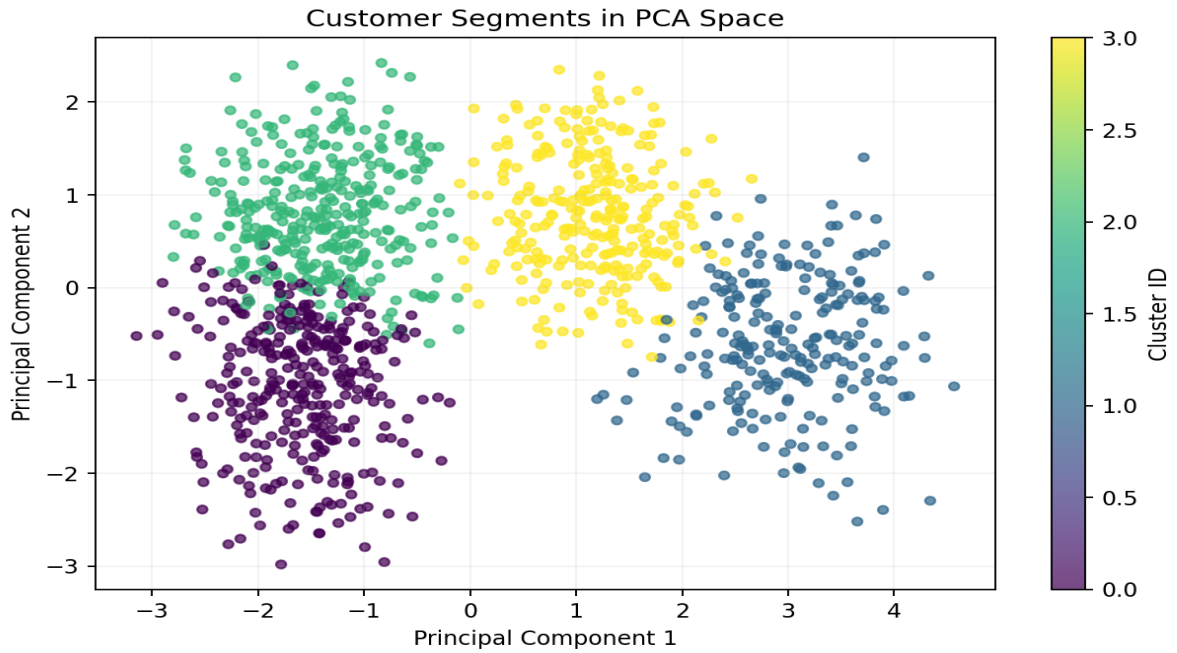


The elbow curve should be evaluated jointly with silhouette scores, cluster stability, profile interpretability, and practical business usefulness. The complete diagnostic table is available at `outputs/cluster_diagnostics.csv`.

4 | Customer segment profiles and interpretation

The final cluster labels are descriptive names generated from the average behavior profiles. They are convenient summaries, not ground-truth customer classes.

Segment	Customers	Share	Income (k)	Spend score	Purchases/mo	AOV
Occasional Value Seekers	336	28.0%	42.9	37.3	2.77	40.8
Premium Loyalists	233	19.4%	104.2	81.6	8.85	134.0
Digital Casuals	349	29.1%	49.0	33.6	2.04	36.7
High-Potential Shoppers	282	23.5%	80.6	70.1	5.66	81.5



Illustrative profile reading

Premium Loyalists: higher average income, order value, spend score and transaction frequency. In a real setting, test retention benefits and avoid over-discounting customers who already buy regularly.

High-Potential Shoppers: above-average spend and frequency, with high online engagement. Consider relevant cross-sell or loyalty tests, then measure incremental lift.

Digital Casuals: high online ratio but comparatively low purchase frequency and order value. Evaluate low-friction repeat-purchase journeys.

Occasional Value Seekers: lower purchase frequency and lower typical order value. Evaluate value messaging and test promotions against margin impact.

5 | Task 4 - Time-series stationarity and SARIMAX forecast

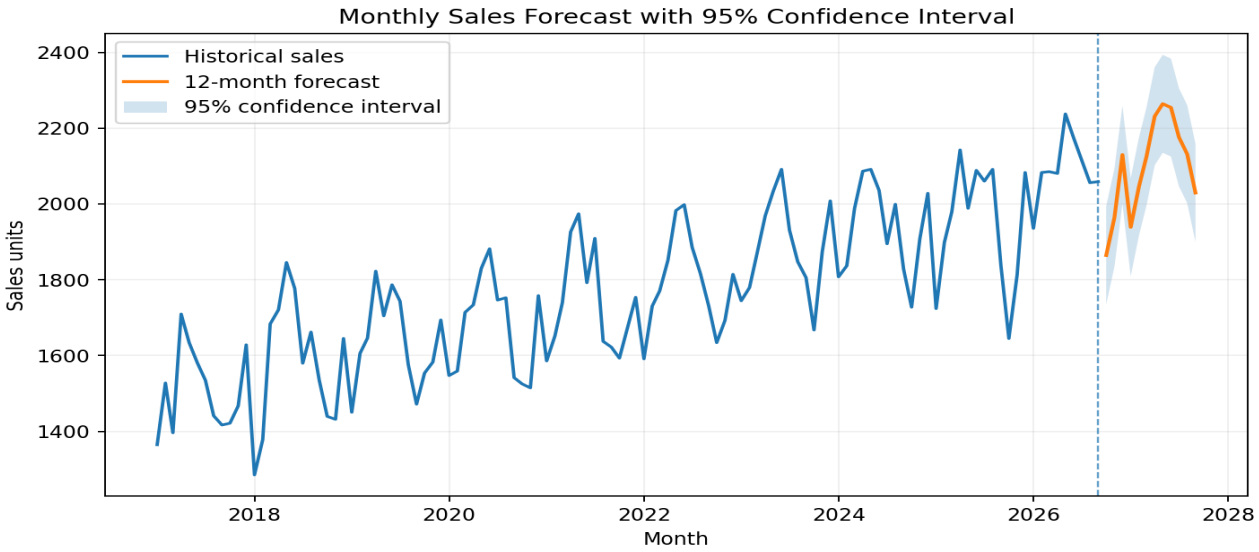
The time series contains 117 synthetic monthly sales observations from January 2017 through September 2026. It includes a gradual trend, recurring annual seasonality, a modest holiday uplift, and seeded noise. It represents a reproducible demonstration, not real company performance.

Stationarity checks

Series	ADF statistic	p-value	Conclusion at 5%
Raw sales	-0.442	0.9028	Fail to reject unit-root null; non-stationary evidence
12-month seasonal difference	-3.136	0.0240	Reject unit-root null; stationary under this test
Seasonal + regular difference (check)	-2.688	0.0761	Does not reject at 5%; differencing choice needs care

The fitted SARIMAX specification is order (1,0,1), seasonal order (1,1,1,12), consistent with seasonal differencing at a 12-month period. The last 12 historical months were held out for validation before refitting to the full series for future forecasting.

Forecast with 95% confidence intervals

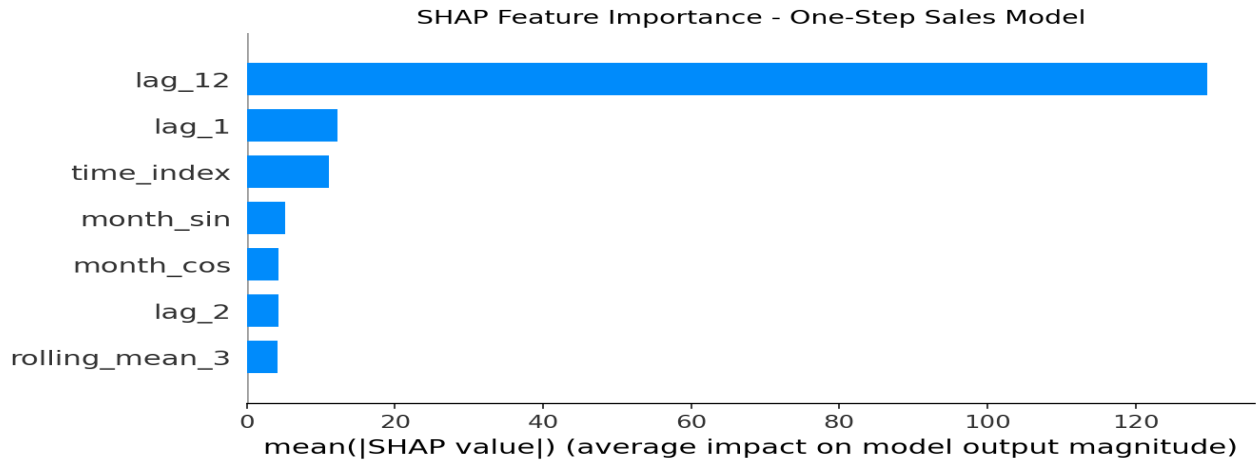


Holdout metrics: MAE = 69.88, RMSE = 89.31, MAPE = 3.63%. Forecast dates: 2026-10-01 to 2027-09-01. Forecast average = 2096.1 sales units per month. Confidence bands communicate model uncertainty and should not be treated as guaranteed limits.

6 | Model interpretability, limitations and reproducibility

SHAP feature contributions

A separate Random Forest one-step-ahead model uses time index, month-of-year encoding, sales lags, and a three-month rolling average. SHAP TreeExplainer calculates per-feature contributions on a sample of 60 rows; average absolute SHAP values summarize the magnitude of contributions in the model output unit.



Feature	Mean absolute SHAP	Interpretation
lag_12	129.64	Same month in the previous year; captures annual seasonality.
lag_1	12.26	Previous month value; captures local persistence.
time_index	11.05	Long-run temporal position; may capture underlying growth.
month_sin	5.14	Calendar season position.
month_cos	4.30	Calendar season position.

Interpretation boundary: SHAP explains the auxiliary Random Forest, not the SARIMAX model itself. Lag_12 is the strongest driver in this demo, which is consistent with the seasonal structure built into the synthetic series; it does not establish a causal effect.

Limitations and responsible use

Synthetic data cannot support real commercial claims. Clipping and median imputation are teaching choices and should be validated against the real data-generating process. Cluster IDs are arbitrary; segment names are heuristic. The scraper must be run online for live records and adapted only after checking source policies. Forecast accuracy can change materially under structural breaks, promotions, stockouts, missing periods, or data revisions. A single train/holdout split is useful for demonstration but rolling-origin validation is preferable for production.

Reproduce

1. Install dependencies: `pip install -r requirements.txt`
2. Regenerate Task 3/4: `python analysis_pipeline.py`
3. Run live scraping: `python task2_web_scraper.py --pages 5 --output-dir outputs/task2_live_scrape`
4. Launch dashboard: `streamlit run dashboard.py`
5. Open `Data_Science_Project_Notebook.ipynb` for a guided walkthrough.

Source target for scraper implementation: Quotes to Scrape, https://quotes.toscrape.com/. All generated numeric results in this whitepaper are reproducible synthetic-demo outputs.